Eerly Workspace AI

System Architecture — 44-Section Engineering Reference

1. Purpose

Eerly Workspace AI is a backend-first AI workspace consultant. It accepts an authenticated user's request, operates only inside that user's workspace, uses a LangGraph workflow to coordinate analysis and generation, pauses for human approval before delivery, and persists execution state and artifact metadata in PostgreSQL.

The architecture separates HTTP/API concerns, authentication, agent orchestration, LLM interaction, filesystem operations, workspace isolation, skills, persistence, artifact delivery, and observability. Security-sensitive and infrastructure-sensitive behavior remains outside the LLM.

2. High-Level Architecture

```
Streamlit (thin frontend)
    |
    | HTTP / SSE
    v
FastAPI
    |
    v
JWT Authentication
    |
    v
Agent Service / LangGraph
    |
    +--> Skills / SKILL.md
    +--> Secure Workspace Tools
    +--> Gemini LLM
    +--> Validation
    +--> Human Approval
              |
              +--> Approve --> Artifact Delivery
              +--> Edit ----> Validation --> Approval
              +--> Reject --> END
    |
    v
PostgreSQL
    +--> Application data
    +--> LangGraph checkpoints
```

3. Architectural Layers

The system is divided into focused layers.

3.1 Frontend Layer — Streamlit collects requests, displays progress and approval controls, sends API requests, and consumes SSE. It does not own authorization, filesystem access, database access, LangGraph execution, or Gemini credentials.

3.2 API Layer — FastAPI handles HTTP, Pydantic validation, authentication dependencies, service calls, typed responses, and SSE.

3.3 Authentication Layer — JWT bearer authentication establishes the trusted user_id.

3.4 Agent Orchestration Layer — LangGraph coordinates the workflow.

3.5 Skills Layer — Discovers, selects, validates, and loads SKILL.md instructions on demand.

3.6 Workspace and Filesystem Layer — Owns secure per-user filesystem access.

3.7 LLM Layer — Separates Gemini interaction from graph orchestration.

3.8 Persistence Layer — SQLAlchemy, PostgreSQL, and Alembic provide application persistence and checkpoint storage.

3.9 Artifact Layer — Delivers approved outputs and persists artifact metadata.

3.10 Observability Layer — Structured JSON logging, correlation IDs, and standardized API errors.

4. API Layer

Primary implementation: app/api/agent.py

Endpoints:
POST /agent/run

POST /agent/resume
GET /agent/stream/{thread_id}

```
Flow:
HTTP request -> Pydantic validation -> JWT authentication -> trusted user_id -> Agent service.
```

The API layer does not implement the LangGraph workflow itself.

5. Authentication Boundary

Authentication is implemented in app/auth/dependencies.py.

```
Authorization: Bearer <token>
        |
        v
JWT verification
        |
        v
JWT sub
        |
        v
trusted user_id
```

The client does not provide the authoritative user_id. The authenticated identity is the basis for workspace selection, filesystem isolation, SSE isolation, checkpoint isolation, and workspace ownership.

6. Agent Layer

Implementation: app/agent/

```
The workflow is:
Request Analysis
 -> Skill Discovery
 -> Skill Loading
 -> Workspace Inspection
 -> Source Reading
 -> Content Analysis
 -> Output Generation
 -> Output Validation
 -> Human Approval
```

Approval routes to delivery, re-validation after edit, or rejection.

7. LangGraph State

Implementation: app/agent/state.py

The state is JSON-safe because it is checkpointed. It contains user/thread identity, request analysis, skills, workspace file metadata, source contents, findings, generated output, artifact metadata, validation, approval state, execution status, and error state.

The state is the durable workflow representation.

8. JSON-Safe State Design

```
Pydantic model
    -> model_dump()
    -> JSON-safe dictionary
    -> LangGraph state
    -> PostgreSQL checkpoint

Checkpoint dictionary
    -> model_validate()
    -> typed application model
```

This avoids unnecessary non-serializable Python objects in durable state.

9. Request Analysis

The first workflow stage interprets the natural-language request and creates structured workflow intent for downstream graph nodes.

10. Skill Discovery

Implementation: app/skills/ and app/agent/nodes/skill_discovery.py

Skills are directories containing SKILL.md. Current skills include document_analysis and report_generation. The system discovers available skills and selects applicable skills for the request.

11. On-Demand Skill Loading

```
Discovery
  -> Select relevant skills
  -> Load selected SKILL.md
```

The loader validates skill names and filesystem containment before loading content. This makes the agent extensible without hard-coding every capability into the graph.

12. Workspace Layer

Implementation: app/workspace/manager.py

Per-user workspace:
workspaces/
user_a/
input/
working/
delivered/
user_b/
input/
working/
delivered/

The authenticated user determines which workspace can be accessed.

13. Filesystem Security Boundary

Implementation: app/tools/filesystem.py with WorkspaceManager.

```
Authenticated user
 -> WorkspaceManager
 -> resolve requested path
 -> normalize filesystem path
 -> containment check
 -> filesystem operation
```

The LLM cannot bypass this boundary.

14. Path Traversal Protection

The manager resolves both workspace root and requested path before checking containment.

candidate = (user_root / relative_path).resolve()
candidate.relative_to(user_root)

This protects against parent traversal, Windows backslash traversal, absolute paths, normalized traversal, cross-user paths, and symlink escapes.

15. Write Restrictions

Reads and writes have different rules.

```
input/       -> read
working/     -> read + write
delivered/   -> read + write
```

The input directory is treated as source material and is not writable through the general workspace write tool.

16. LLM Layer

LLM integration is separated from graph orchestration. Relevant code is under app/llm/, app/agent/llm.py, and app/agent/structured_llm.py.

```
Streamlit -> FastAPI -> Backend LLM layer -> Gemini API
```

The Gemini credential remains on the backend.

17. Structured LLM Output

Where structured output is required:

```
LLM response
 -> structured parsing
 -> Pydantic validation
 -> application logic
```

This reduces malformed model output propagating through the workflow.

18. Human Approval Boundary

Implementation: app/agent/nodes/human_approval.py

```
Output Generation
 -> Validation
 -> Prepare Approval
 -> interrupt()
 -> persisted checkpoint
 -> WAITING
```

Approval is therefore a durable workflow state rather than a frontend-only confirmation.

19. Approval Decisions

Supported decisions:
approve
edit
reject

```
approve -> delivery
edit -> validation -> approval
reject -> END
```

An edited result must be validated again and receive final approval.

20. Durable Checkpointing

Implementation: app/agent/checkpoint.py

PostgreSQL stores:
- application data
- LangGraph checkpoint data

This allows interrupted workflow state to survive recreation of graph and checkpointer objects.

21. Checkpoint Isolation

The client-visible thread ID is namespaced internally:

effective_thread_id = user_id + ":" + thread_id

Examples:
user_a:report-123
user_b:report-123

The same namespacing is used for starting, retrieving, resuming, and SSE stream keys.

22. Artifact Delivery

Implementation: app/artifacts/service.py

Approved generated output is delivered to the authenticated user's delivered/ directory. Artifact metadata is persisted in PostgreSQL, including artifact ID, workspace, filename, relative path, type, status, and creation timestamp.

Artifact paths are unique within a workspace.

23. SSE Progress Streaming

Endpoint: GET /agent/stream/{thread_id}

Implementation:
app/agent/events.py

app/agent/event_bus.py

```
LangGraph node -> Agent service -> Event bus -> SSE subscriber -> Streamlit
```

Events include request_received, request_analyzed, skills_discovered, skills_loaded, workspace_inspected, sources_read, analysis_completed, output_generated, validation_completed, approval_required, approval_received, artifact_delivered, completed, rejected, and failed.

Heartbeat comments keep long-running connections active.

24. SSE Isolation

SSE stream keys use user_id:thread_id.

user_a:thread-1
and
user_b:thread-1

are different event streams, preventing cross-user event subscription by thread ID alone.

25. Event Bus Trade-off

The current event bus is process-local, which is appropriate for the single-process assessment deployment.

For horizontally scaled API replicas, a shared event infrastructure such as Redis Streams or Kafka would be appropriate.

26. Database Layer

Implementation: app/db/

```
engine.py       -> database engine
session.py      -> async session factory
dependencies.py -> FastAPI database dependency
models/         -> SQLAlchemy models
repository/     -> database access logic
base.py         -> declarative base
```

The application uses asynchronous SQLAlchemy with PostgreSQL.

27. Database Schema

Main relationship:

```
User
  |
  | 1:1
  v
Workspace
  |
  | 1:N
  v
Artifact
```

users: id, created_at
workspaces: id, user_id, created_at
artifacts: id, workspace_id, filename, relative_path, artifact_type, status, created_at

28. Migrations

Alembic manages schema changes.

Docker startup executes:
alembic upgrade head

The migration history creates the application tables and artifact path uniqueness constraint.

29. Observability

Implementation: app/core/logging.py

Logs are emitted as structured JSON containing fields such as timestamp, level, logger, message, correlation_id, HTTP method/path/status, and duration.

30. Correlation IDs

Implementation:
app/core/correlation.py
app/core/middleware.py

```
Incoming request
  -> use X-Correlation-ID if supplied
  -> otherwise generate UUID
  -> store in ContextVar
  -> include in logs
  -> return in response header
```

This allows request tracing across API responses and logs.

31. Error Handling

Client input errors return structured error, message, and correlation_id fields.

Unexpected application errors return a generic client-facing message while detailed exception information is logged server-side. This prevents internal database, credential, filesystem, or stack-trace information from leaking through the API.

32. Health Checks

API endpoints:
GET /health
GET /health/db
GET /health/db/session

Docker also uses healthchecks and waits for PostgreSQL to become healthy before starting the API.

33. Container Architecture

Docker Compose provides:

```
PostgreSQL
   |
   | named volume: postgres_data
   |
   +--> healthy
           |
           v
         API
```

The API container runs Alembic migrations before Uvicorn.

34. Complete Execution Flow

```
User
  -> Streamlit
  -> POST /agent/run
  -> FastAPI
  -> JWT Authentication
  -> trusted user_id
  -> workspace provisioning
  -> LangGraph
  -> request analysis
  -> skill discovery
  -> skill loading
  -> workspace inspection
  -> source reading
  -> content analysis
  -> output generation
  -> validation
  -> human approval
  -> approve/edit/reject
  -> artifact delivery when approved
```

35. Data and Control Flow

Control flow is owned by LangGraph:

```
Request -> Analysis -> Generation -> Validation -> Approval -> Delivery
```

Data flow:

```
User Request
  -> Pydantic Schema
  -> LangGraph State
  -> Skills / Workspace Tools / Gemini / Validation / Human Approval
  -> Artifact
```

```
  -> PostgreSQL metadata
```

Separating control and data flow makes the system easier to reason about.

36. Security Architecture

Security is defense in depth:

JWT authentication
+
safe user ID validation
+
per-user workspace mapping
+
path containment
+
write restrictions
+
checkpoint namespacing
+
SSE namespacing
+
generic API errors

The LLM is never treated as the security boundary.

37. Failure and Recovery Model

Execution can reach:

```
RUNNING
  |
  v
WAITING_FOR_APPROVAL
  |
  v
PostgreSQL checkpoint
```

If the original graph/checkpointer objects disappear, new instances can resume using the same effective thread ID.

The real PostgreSQL durability test closes the original objects, creates new ones, and resumes the interrupted execution.

38. Idempotency Considerations

LangGraph nodes may execute again when an interrupted workflow resumes. Side effects should therefore be carefully controlled around the approval boundary.

Analysis, generation, and validation are separated from final artifact delivery. Delivery occurs only after approval.

39. Project Structure

```
eerly-workspace-ai/
■■■ app/
■   ■■■ agent/
■   ■   ■■■ nodes/
■   ■   ■■■ prompts/
■   ■   ■■■ checkpoint.py
■   ■   ■■■ events.py
■   ■   ■■■ event_bus.py
■   ■   ■■■ graph.py
■   ■   ■■■ llm.py
■   ■   ■■■ models.py
■   ■   ■■■ service.py
■   ■   ■■■ state.py
■   ■   ■■■ structured_llm.py
■   ■■■ api/
■   ■■■ artifacts/
■   ■■■ auth/
■   ■■■ core/
■   ■■■ db/
■   ■■■ llm/
■   ■■■ schemas/
■   ■■■ skills/
■   ■■■ tools/
■   ■■■ workspace/
■   ■■■ main.py
■■■ skills/
■   ■■■ document_analysis/SKILL.md
■   ■■■ report_generation/SKILL.md
■■■ migrations/
■■■ tests/
■■■ docs/
■■■ Dockerfile
■■■ docker-compose.yml
■■■ entrypoint.sh
■■■ alembic.ini
■■■ pyproject.toml
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

40. Core Architectural Principles

Security at boundaries — security-sensitive behavior is enforced outside the LLM.

Durable state — long-running execution is represented as checkpointed state.

Explicit human control — final delivery requires explicit approval.

Separation of concerns — API, agent orchestration, tools, persistence, authentication, and observability have focused responsibilities.

Defense in depth — multiple independent controls protect user isolation.

Explainability — focused modules and explicit behavior make the system easier to inspect, test, and defend.

41. Production Scaling Considerations

The assessment implementation intentionally keeps some infrastructure simple.

Event infrastructure: replace the process-local event bus with shared infrastructure.

Workspace storage: move to shared/object storage where appropriate while preserving tenant isolation.

Authentication: integrate with an enterprise identity provider and key rotation.

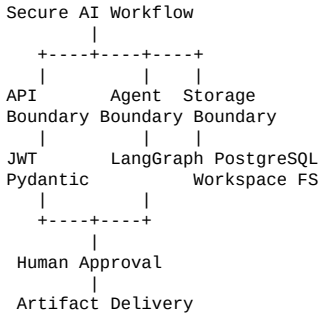
Observability: centralize logs and metrics.

API scaling: run multiple replicas behind a load balancer.

Database: use managed PostgreSQL, connection-pool sizing, backups, migration controls, and monitoring.

Secrets: use a dedicated secrets manager rather than environment files for production credentials.

42. Architecture Summary

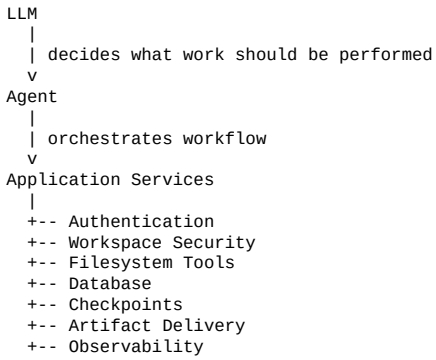


Central principle: the LLM coordinates work, but does not own security, persistence, authorization, or filesystem boundaries.

43. Architecture Decision Summary

API: FastAPI
Agent orchestration: LangGraph
LLM: Gemini
Database: PostgreSQL
ORM: SQLAlchemy
Migrations: Alembic
Checkpointing: PostgreSQL-backed LangGraph checkpointer
Authentication: JWT bearer authentication
Workspace isolation: per-user filesystem directories
Filesystem security: path resolution + containment validation
Skills: on-demand SKILL.md discovery/loading
Human approval: LangGraph interrupt/resume
Streaming: Server-Sent Events
Event transport: in-process event bus
Artifact storage: user delivered workspace + PostgreSQL metadata
Frontend: Streamlit
Containerization: Docker Compose
Logging: structured JSON
Request tracing: correlation IDs
Testing: pytest

44. Final Architectural Principle



The AI model is a component of the system, not the system's security or persistence layer.

This separation makes the application easier to test, secure, recover, scale, and explain during an engineering review.